

Project Title: Ski Resort Gear Rental System
Course: CS340
Group: 80
Team Members: Joshua Cabalka, Jack Boland, Philip Gadsden
Project Step 3 Draft Version: Design UI Interface + DML SQL
Date: 02/12/2026

a) Project URL: <http://classwork.engr.oregonstate.edu:40405>

b) Feedback from Step 3

Feedback by the peer reviewer:
Thane Thompson

Group 80, My response in BOLD

- o A SELECT for every table in the schema? YES
- o Does the UI implement an INSERT form for at least one table in the schema? YES
- o Does the UI have at least one DELETE for any one entity? DELETE APPEARS PRESENT ON THE UI,
BUT IT DOES NOT DELETE RECORDS
- o Does the UI have at least one DELETE that will remove things from a M:M relationship? DELETE DOES
NOT REMOVE THINGS
- o Is there at least one UPDATE form in the UI for any one entity? YES, IT WORKS ON CUSTOMER
- o Is there at least one UPDATE form in the UI to modify an M:M relationship? YES IS PRESENT ON UI,
BUT DOES NOT WORK FOR RENTAL ORDER ITEMS
- o Do you have any other suggestions for the team to help with their HTML UI? EVENTUALLY, THE USER
WILL NOT HAVE TO SELECT THE VARIOUS IDs TO UPDATE DATA
- o As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original ALL ME.

Cadin Martin

To start off, all the written work here is my own, I did not use any AI to assist in this review. I see the Select system is working, I can clearly see all of the inserted sample data; there's also a clear area for INSERT, DELETE, and UPDATE for all the pages, with pseudo-functionality looking ready to go.

Overall, it looks well-done and well-crafted. If I were to suggest anything, it's stylistic rather than functional, moving into the Part 3 Final: Enlarging and centering some of the ADD/DELETE/UPDATE sections could

make it look more professional, and adding a link between each page to the other pages helps save time (Right now, I have to go back to the index to go to any of the other pages, which slows things down).

Ethan Mark

Does the UI utilize a SELECT for every table in the schema? In other words, data from each table in the schema should be displayed on the UI. Note: it is generally not acceptable for just a single query to join all tables and displays them.

Yes, the UI has a select query for every table in the schema

Does the UI implement an INSERT form for at least one table in the schema? In other words, there should be

UI input fields that correspond to at least one table.

Yes, each table has an INSERT form for each table

Does the UI have at least one DELETE for any one entity? In other words, is there a form/link/button that will

allow the deletion of a row in at least one table?

Yes, each table has a DELETE for every entity

Does the UI have at least one DELETE that will remove things from a M:M relationship? In other words, if an

order is deleted from the Orders table, it should also delete the corresponding rows from the OrderDetails table,

BUT it should not delete any Products or Customers.

Yes, the UI has a DELETE form for M:M relationship between Gear items & Rental Order Items

Is there at least one UPDATE form in the UI for any one entity? In other words, in the case of Products, can

productName, listPrice, qtyOnHand, e.g. be updated for a single ProductID record?

Yes, each table has an UPDATE form to update records within the table

Is there at least one UPDATE form in the UI to modify an M:M relationship? In other words, does the

UPDATE allow the user to select a different foreign key value to update the intersection table with

Yes, there is an UPDATE form for M:M relationship Rental Order Items & Gear Items

Do you have any other suggestions for the team to help with their HTML UI? For example, using AS aliases to

replace obscure column names such as fname with First Name.

I think that having to click 'Back to Index' is a bit unintuitive and having just the page selector at the top on any

page will make the user experience much cleaner

Nicole Rem Pingul

Hi Group 80,

The UI utilizes a SELECT for every table in the schema since each page has a table showing

sample
data

The UI implements an INSERT form for Customers, Employees, Gear Items, Rental Orders, Service

Tickets, and Service Ticket Items

The UI has DELETE form for every entity and for M:M relationship between Rental Order Items and Gear Items.

There is an UPDATE form for each entity and for M:M relationship between Rental Order Items and Gear Items.

I think it would be nice to incorporate a navigation bar at the top of each page or on the side to easily navigate between pages without going back to the home page every time a user would like to

view another page. I also noticed that you guys were missing a ERD in your PDF.

Also, I would suggest using a JOIN for Rental Orders to incorporate customer's names and employee names for the tables. It would be easier to understand the data provided that way. You can

also apply it the Service Ticket Items and Rental Order Items with the gear items as well.

Great job with your guys' website so far!

Good luck with the rest of your project

c) Fixes Based on Feedback from Step 3

- i) Added a shared navigation header on all pages to make it easier for users to move between pages without having to return to the index.
- ii) DELETE operations properly implemented.

d) Design Changes

- i) Added DB reset stored procedure in its own SQL file. The DML file calls it when the UI frontend requests it. There is a single reset button on the main index.html page that will reset the database to just the example data.
- ii) Converted all CRUD operations into stored procedures and stored them in a separate SQL file. Updated DML file to call procedures from the procedures file.
- iii) Fixed run_action function to safely handle stored procedure calls by draining results set to avoid errors.
- iv) Fixed UI schema enum mismatches.

e) Fixes Based on Feedback from Step 2

Feedback based on the Step 2 submission was positive, with many reviewers noting that the system design, entities, relationships, ERD and Schema were clear and aligned with project requirements. No major changes were made to the Schema or SQL.

f) Fixes Based on Feedback from Step 1

Feedback on the Step 1 submission was mainly positive, with reviewers agreeing that the system design, entities, relationships, and ERD were clear and aligned with requirements. Only minor issues were identified and addressed.

Reviewers noted that the overview could better illustrate system scale. The overview was updated to more explicitly describe operational scope, including individual gear tracking, multi-item rental orders, historical checkout and return records, and independent maintenance workflows.

An error was identified in the database outline where `customer_id` in the `rental_orders` table incorrectly referenced the `employees` table. This was corrected to reference `customers(customer_id)`, aligning the outline with the ERD and business logic.

A minor naming inconsistency was identified in the `rental_order_items` table, where the foreign key name did not match the rental order primary key naming convention. Fixed foreign key reference so `rental_orders.customer_id` correctly references `customers(customer_id)`.

Clarified meaning of `employees.is_active` (employee is currently employed/authorized to create orders and service tickets).

No other changes were required, as reviewers confirmed that entity modeling, relationships, naming conventions, and the ERD were correct.

g) Project Overview and Database Outline

This project models an admin-facing ski resort gear rental system designed to manage an equipment inventory of approximately 300 individual gear items across various categories, along with an average of 80 rental transactions per week and 20 maintenance service tickets per week.

The system tracks individual gear items to allow staff to monitor availability, condition, rental history, and service history at the item level. Rental activity is recorded through rental orders created by employees for customers, with each rental order capable of containing multiple gear items. A typical rental order contains 2–4 gear items. Historical rental data is preserved using timestamps that record when items are checked out and returned.

Maintenance and repair work is tracked independently using service tickets, which document service procedures performed on one or more gear items. Each service ticket includes timestamps for work start and completion, allowing the system to maintain a complete maintenance history separate from rental activity. Together, these components support both daily operational needs and long-term inventory and service tracking for the ski resort.

DATABASE OUTLINE

1. **customers:** Records customers who rent gear from the ski resort.

Attributes

- `customer_id`: INT, AUTO_INCREMENT, PRIMARY KEY, NOT NULL

- first_name: VARCHAR(100), NOT NULL
- last_name: VARCHAR(100), NOT NULL
- phone: VARCHAR(20), NULL
- email: VARCHAR(255), NULL

Relationships

- One-to-many (1:M) relationship with rental_orders

2. **employees:** Records employees who create rental orders and service tickets.

Attributes

- employee_id: INT, AUTO_INCREMENT, PRIMARY KEY, NOT NULL
- first_name: VARCHAR(100), NOT NULL
- last_name: VARCHAR(100), NOT NULL
- role: VARCHAR(50), NOT NULL
- is_active: BOOLEAN, NOT NULL, DEFAULT TRUE
- Indicates whether the employee is currently employed and authorized to create rental orders and service tickets

Relationships

- One-to-many (1:M) relationship with rental_orders
- One-to-many (1:M) relationship with service_tickets

3. **gear_items:** Records each physical piece of ski or snowboard-related equipment.

Attributes

- gear_item_id: INT, AUTO_INCREMENT, PRIMARY KEY, NOT NULL
- category: ENUM('snowboard', 'boots', 'helmet', 'other'), NOT NULL
- brand: VARCHAR(100), NOT NULL
- model: VARCHAR(100), NOT NULL
- serial_number: VARCHAR(100), UNIQUE, NOT NULL
- size: VARCHAR(20), NOT NULL
- condition_grade: ENUM('new', 'good', 'fair', 'needs_repair'), NOT NULL
- status: ENUM('available', 'rented', 'in_service', 'retired'), NOT NULL
- acquired_at: DATETIME, NOT NULL

Relationships

- Many-to-many (M:M) relationship with rental_orders
- Many-to-many (M:M) relationship with service_tickets

4. **rental_orders:** Records each rental checkout transaction.

Attributes

- rental_order_id: INT, AUTO_INCREMENT, PRIMARY KEY, NOT NULL
- customer_id: INT, NOT NULL, FOREIGN KEY → customers(customer_id)

- created_by_employee_id: INT, NOT NULL, FOREIGN KEY → employees(employee_id)
- created_at: DATETIME, NOT NULL

Relationships

- Many-to-one (M:1) relationship with customers
- Many-to-one (M:1) relationship with employees
- Many-to-many (M:M) relationship with gear_items (via rental_order_items)

5. **service_tickets**: Records maintenance and repair work performed on gear items.

Attributes

- service_ticket_id: INT, AUTO_INCREMENT, PRIMARY KEY, NOT NULL
- opened_by_employee_id: INT, NOT NULL, FOREIGN KEY → employees(employee_id)
- status: ENUM('open', 'in_progress', 'completed', 'canceled'), NOT NULL
- created_at: DATETIME, NOT NULL

Relationships

- Many-to-one (M:1) relationship with employees
- Many-to-many (M:M) relationship with gear_items (via service_ticket_items)

INTERSECTION TABLES

6. **rental_order_items**: Implements the many-to-many relationship between rental_orders and gear_items.

Attributes

- rental_order_id: INT, NOT NULL, FOREIGN KEY → rental_orders(rental_order_id)
- gear_item_id: INT, NOT NULL, FOREIGN KEY → gear_items(gear_item_id)
- checked_out_at: DATETIME, NOT NULL
- due_at: DATETIME, NOT NULL
- returned_at: DATETIME, NULL

Primary Key

- (rental_order_id, gear_item_id)

Business Rule

- A gear item may only appear in one active rental at a time. This is enforced by checking that gear_items.status = 'available' prior to checkout and updating the status to 'rented' while returned_at is NULL.

7. **service_ticket_items**: Implements the many-to-many relationship between service_tickets and gear_items.

Attributes

- service_ticket_id: INT, NOT NULL, FOREIGN KEY → service_tickets(service_ticket_id)
- gear_item_id: INT, NOT NULL, FOREIGN KEY → gear_items(gear_item_id)
- service_type: ENUM('wax', 'tune', 'repair', 'binding_adjust', 'inspect'), NOT NULL
- started_at: DATETIME, NULL
- completed_at: DATETIME, NULL

Primary Key

- (service_ticket_id, gear_item_id)

h) Entity-Relationship Diagram (ERD)

i) Schema Diagram + Normalization (Write-up)

The following report was generated by joining rental orders, customers, employees, and gear items into a single flattened result set, where each row represents a gear item within a rental

	rental_order_id	order_created_at	customer_name	customer_phone	customer_email	employee_name	employee_role
1	1	2026-02-01 10:00:00	John Doe	541-555-0101	john.doe@example.com	Shaun White	rental_associate
2	1	2026-02-01 10:00:00	John Doe	541-555-0101	john.doe@example.com	Shaun White	rental_associate
3	1	2026-02-01 10:00:00	John Doe	541-555-0101	john.doe@example.com	Shaun White	rental_associate
4	2	2026-02-02 11:15:00	Noah Patel	<null>	noah.patel@example.com	Shaun White	rental_associate
5	3	2026-02-03 09:20:00	Mia Johnson	541-555-0103	<null>	Shaun White	rental_associate

order. This report reflects how the data might appear in a spreadsheet prior to normalization.

FIGURE 2: UNNORMALIZED RENTAL ACTIVITY SAMPLE REPORT DEMONSTRATING DATA REDUNDANCY AND UPDATE/INSERT/DELETE ANOMALIES PRIOR TO NORMALIZATION.

	serial_number	category	brand	model	size	checked_out_at	due_at	returned_at
1	SNB-0001	snowboard	Burton	Process	155	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
2	BTS-0101	boots	K2	Mysis	10	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
3	HLT-0201	helmet	Smith	Vantage	M	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
4	SNB-0001	snowboard	Burton	Process	155	2026-02-02 11:20:00	2026-02-04 11:20:00	2026-02-03 16:45:00
5	SNB-0001	snowboard	Burton	Process	155	2026-02-03 09:25:00	2026-02-05 09:25:00	<null>

In the sample report, order-level, customer, and employee attributes are repeated across multiple rows because each row represents a single gear item within a rental order. For example, rental order 1 appears three times, resulting in duplicated customer and employee data. This redundancy introduces update anomalies (e.g., a change to a customer's contact information must be applied to multiple rows), insert anomalies (new customers, gear items, or

employees cannot be added independently without creating a full rental transaction), and delete anomalies (removing the final line item for an order may unintentionally remove associated order or customer information).

To convert the design to First Normal Form (1NF), repeating groups were eliminated by separating rental order data from individual gear items. Each rental order may contain multiple gear items, so a separate intersection table was introduced to represent one row per gear item per order. The composite key (`rental_order_id`, `gear_item_id`) uniquely identifies each row. With atomic attributes and no repeating groups, the design satisfies 1NF.

In the 1NF design, partial dependencies existed because order-level attributes depended only on `rental_order_id`, while gear attributes depended only on `gear_item_id`. To convert the design to Second Normal Form (2NF), these partial dependencies were removed by decomposing the schema into `rental_orders`, `gear_items`, and `rental_order_items`. Each non-key attribute now depends on the entire primary key of its table.

Transitive dependencies were identified where customer and employee attributes depended on non-key identifiers in rental orders (e.g., `rental_order_id` $\rightarrow$ `customer_id` $\rightarrow$ customer attributes). These dependencies were eliminated by moving customer information into a customers table and employee information into an employees table, referenced by foreign keys. After this decomposition, all non-key attributes depend only on the primary key of their respective tables, and the schema satisfies Third Normal Form (3NF).

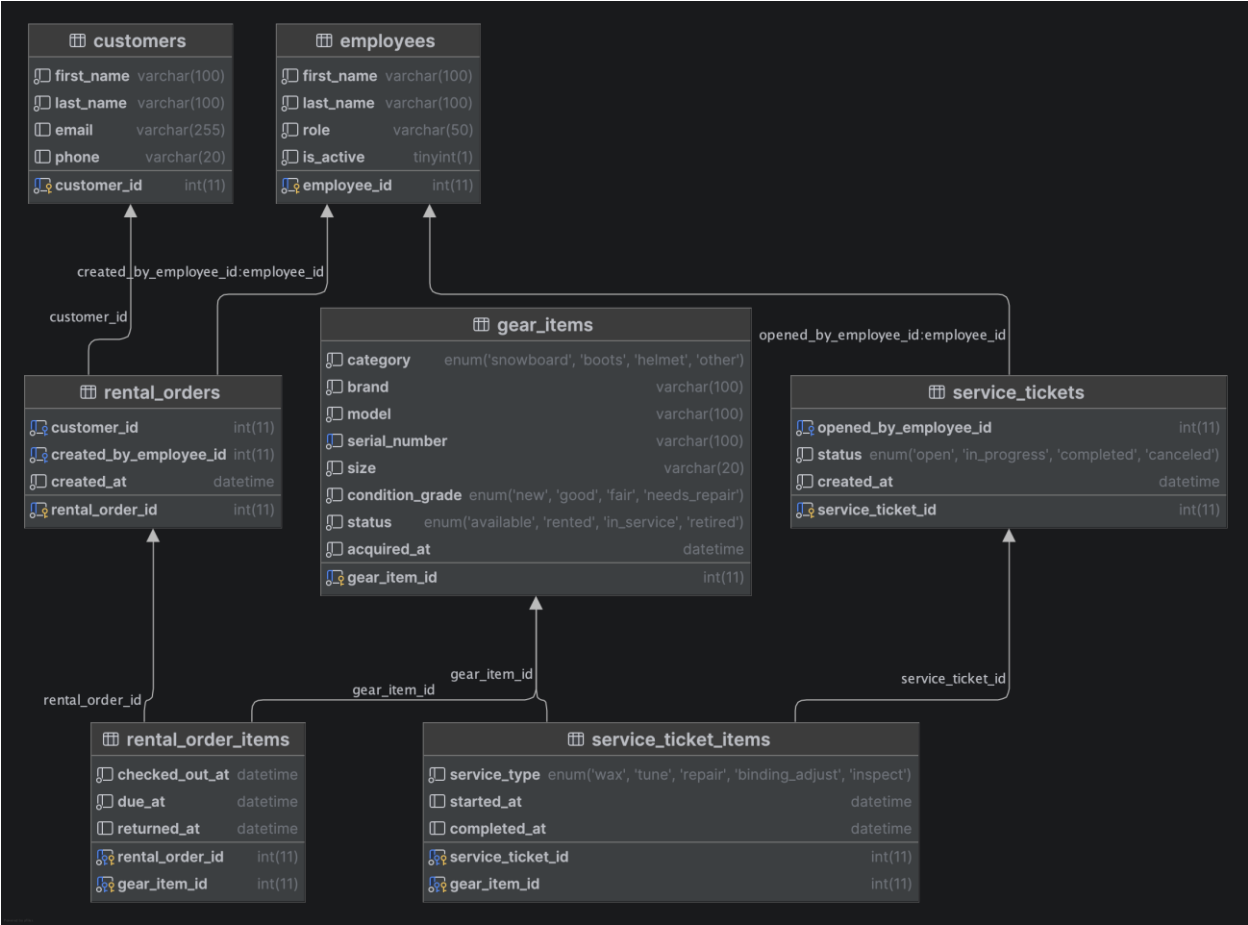


FIGURE 3: FINAL NORMALIZED DATABASE SCHEMA FOR THE SKI RESORT GEAR RENTAL SYSTEM.

j) Example Data

customers

	customer_id	first_name	last_name	email	phone
1	1	John	Doe	john.doe@example.com	541-555-0101
2	2	Noah	Patel	noah.patel@example.com	<null>
3	3	Mia	Johnson	<null>	541-555-0103

employees

	customer_id	first_name	last_name	email	phone
1	1	John	Doe	john.doe@example.com	541-555-0101
2	2	Noah	Patel	noah.patel@example.com	<null>
3	3	Mia	Johnson	<null>	541-555-0103

	employee_id	first_name	last_name	role	is_active
1	1	Shaun	White	rental_associate	1
2	2	Travis	Rice	service_tech	1
3	3	Lindsey	Vonn	manager	0

	gear_item_id	category	brand	model	serial_number	size	condition_grade	status	acquired_at
1	1	snowboard	Burton	Process	SNB-0001	155	good	available	2024-11-01 09:00:00
2	2	boots	K2	Mysis	BTS-0101	10	fair	rented	2023-12-10 10:30:00
3	3	helmet	Smith	Vantage	HLT-0201	M	new	in_service	2024-10-15 14:00:00
4	4	other	Dakine	Wrist Guard	OTH-0301	L	good	retired	2022-01-05 08:15:00

gear_items

	rental_order_id	customer_id	created_by_employee_id	created_at
1	1	1	1	2026-02-01 10:00:00
2	2	2	2	2026-02-02 11:15:00
3	3	3	3	2026-02-03 09:20:00

rental_orders
rental_order_items

	rental_order_id	gear_item_id	checked_out_at	due_at	returned_at
1	1	1	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
2	1	2	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
3	1	3	2026-02-01 10:05:00	2026-02-03 10:05:00	<null>
4	2	1	2026-02-02 11:20:00	2026-02-04 11:20:00	2026-02-03 16:45:00
5	3	1	2026-02-03 09:25:00	2026-02-05 09:25:00	<null>

service_ticket_items

	service_ticket_id ▾	gear_item_id ▾	service_type ▾	started_at ▾	completed_at ▾
1	1	3	inspect	<null>	<null>
2	2	1	binding_adjust	2026-02-02 13:25:00	<null>
3	2	3	repair	2026-02-02 13:15:00	<null>
4	3	3	wax	2026-02-03 15:20:00	2026-02-03 16:05:00

	service_ticket_id ▾	opened_by_employee_id ▾	status ▾	created_at ▾
1	1	2	open	2026-02-01 08:30:00
2	2	2	in_progress	2026-02-02 13:00:00
3	3	2	completed	2026-02-03 15:10:00

service_tickets